

HOSPITAL MANAGEMENT SYSTEM

Program Documentation

Document Type	Program Documentation
Version	1.0
Date	04 May 2026
Language	Java 17+ (Single-file, 1,051 lines)
Database	MySQL 8.x via JDBC
Entry Point	HospitalManagementSystem.main()

1. File & Class Structure

The entire system is contained in a single top-level class `HospitalManagementSystem.java`. All domain models, constants, and methods are defined as static members or static inner classes, enabling simple single-file compilation with no build tool required.

Section	Line Range (approx.)	Contents
Constants & Connection	1 – 40	DB_URL, DB_USER, DB_PASS, getConnection()
Domain Models	41 – 190	Patient, Doctor, Appointment, Bed, Prescription, Invoice, Staff
initDatabase()	191 – 260	CREATE TABLE IF NOT EXISTS for all 8 tables
Patient Service	261 – 370	registerPatient, listPatients, searchPatient, updatePatient, deletePatient
Doctor Service	371 – 430	addDoctor, listDoctors, deleteDoctor
Appointment Service	431 – 510	bookAppointment, listAppointments, updateAppointmentStatus
Bed / Ward Service	511 – 580	addBed, listBeds, admitPatient, dischargePatient
Prescription Service	581 – 660	addPrescription, listPrescriptions, searchPrescriptionsByPatient
Billing Service	661 – 750	createInvoice, listInvoices, recordPayment
Staff Service	751 – 810	addStaff, listStaff, deleteStaff
Reports	811 – 860	showDashboard, todayAppointments
Menu Methods	861 – 980	patientMenu, doctorMenu, appointmentMenu, bedMenu, prescriptionMenu, billingMenu, staffMenu
main()	981 – 1051	Driver load, initDatabase(), main navigation loop

2. Domain Model Reference

2.1 Patient

Field	Description
int id	Auto-increment primary key from DB
String name	Full name of the patient
String phone	Unique contact number
String email	Email address (optional)
String bloodGroup	ABO/Rh blood group (e.g. O+)
String address	Physical address
LocalDate dob	Date of birth

2.2 Doctor

Field	Description
int id	Auto-increment primary key
String name	Doctor's full name
String specialization	Medical specialty (e.g. Cardiology)
String phone	Contact number
String email	Email address
double consultationFee	Per-visit fee in local currency (KES)

2.3 Appointment

Field	Description
int id	Primary key
int patientId / patientName	FK reference + display name
int doctorId / doctorName	FK reference + display name
LocalDateTime dateTime	Scheduled date and time
String status	SCHEDULED COMPLETED CANCELLED
String notes	Clinical or admin notes

2.4 Bed

Field	Description
-------	-------------

int id	Primary key
String ward	Ward name (e.g. Maternity, ICU)
String type	General ICU Private
boolean occupied	TRUE if currently in use
Integer patientId	Nullable FK to patients
String patientName	Denormalized display field

2.5 Prescription

Field	Description
int id	Primary key
int appointmentId	Optional FK to appointments
int patientId	FK to patients
String medication	Drug name
String dosage	e.g. 500mg twice daily
String instructions	Special instructions
LocalDate prescribedDate	Date of prescription (defaults to today)

2.6 Invoice

Field	Description
int id	Primary key
int patientId	FK to patients
double totalAmount	Sum of all invoice_items
double paidAmount	Cumulative payments received
String status	PENDING PARTIAL PAID
LocalDate issueDate	Date created

2.7 Staff

Field	Description
int id	Primary key
String name	Employee full name
String role	Job title (Nurse, Admin, etc.)
String department	Organisational unit
String phone / email	Contact details

double salary	Monthly salary in KES
---------------	-----------------------

3. Method Reference

3.1 Database Infrastructure

Method Signature	Throws	Returns	Description
getConnection()	SQLException	Connection	Returns or creates the singleton JDBC connection. Double-checked locking ensures thread-safety.
initDatabase()	SQLException	void	Executes CREATE TABLE IF NOT EXISTS for all 8 tables. Called once at startup.

3.2 Patient Service

Method Signature	Throws	Returns	Description
registerPatient(Scanner)	SQLException	void	Prompts for all patient fields and executes a parameterised INSERT. Prints generated patient_id.
listPatients()	SQLException	void	Executes SELECT * FROM patients and prints each row as a Patient.toString().
searchPatient(Scanner)	SQLException	void	Accepts a search string and performs LIKE query on name and phone columns.
updatePatient(Scanner)	SQLException	void	Builds a dynamic UPDATE statement, only setting columns for non-blank inputs.
deletePatient(Scanner)	SQLException	void	Deletes a patient by ID. FK constraints may prevent deletion if linked records exist.

3.3 Doctor Service

Method Signature	Throws	Returns	Description
addDoctor(Scanner)	SQLException	void	Inserts a new doctor record including specialization and consultation fee.
listDoctors()	SQLException	void	Lists all doctors ordered by doctor_id.
deleteDoctor(Scanner)	SQLException	void	Removes a doctor by ID. Will fail with FK error if appointments exist.

3.4 Appointment Service

Method Signature	Throws	Returns	Description
bookAppointment(Scanner)	SQLException	void	Checks for slot conflicts before inserting. Calls listPatients() and listDoctors() inline for reference.
listAppointments()	SQLException	void	JOINS appointments with patients and doctors; displays full records ordered by datetime DESC.

updateAppointmentStatus(Scanner)	SQLException	void	Updates status to SCHEDULED, COMPLETED, or CANCELLED for a given appointment_id.
todayAppointments()	SQLException	void	Uses DATE(appt_datetime) = CURDATE() to filter only today's records.

3.5 Bed / Ward Service

Method Signature	Throws	Returns	Description
addBed(Scanner)	SQLException	void	Inserts a new bed with ward name and type. Defaults occupied to FALSE.
listBeds()	SQLException	void	LEFT JOINS beds with patients to show occupant name for occupied beds.
admitPatient(Scanner)	SQLException	void	UPDATE with condition occupied=FALSE ensures double-admission is impossible.
dischargePatient(Scanner)	SQLException	void	Sets occupied=FALSE and patient_id=NULL, returning the bed to the available pool.

3.6 Prescription Service

Method Signature	Throws	Returns	Description
addPrescription(Scanner)	SQLException	void	Inserts prescription with optional appointment_id FK (NULL if 0 entered).
listPrescriptions()	SQLException	void	Full JOIN with patients and doctors; ordered by prescribed_date DESC.
searchPrescriptionsByPatient(Scanner)	SQLException	void	Filters prescriptions WHERE patient_id=? for a targeted patient history view.

3.7 Billing Service

Method Signature	Throws	Returns	Description
createInvoice(Scanner)	SQLException	void	Creates the invoice header, then loops collecting line items until 'done'. Sums and updates total.
listInvoices()	SQLException	void	JOINS invoices with patients; shows all invoices with amounts and status.
recordPayment(Scanner)	SQLException	void	Fetches current paid_amount, adds new payment, recalculates status PAID or PARTIAL.

3.8 Staff Service

Method Signature	Throws	Returns	Description
addStaff(Scanner)	SQLException	void	Inserts a staff member with role, department, and salary information.
listStaff()	SQLException	void	Lists all staff ordered by department then name.

deleteStaff(Scanner)	SQLException	void	Removes a staff record by staff_id.
----------------------	--------------	------	-------------------------------------

3.9 Reports

Method Signature	Throws	Returns	Description
showDashboard()	SQLException	void	Executes 8 aggregate queries (COUNT, SUM) and displays a formatted summary panel.
todayAppointments()	SQLException	void	Filtered appointment listing for the current calendar date.

4. Key SQL Statements

All SQL is executed via PreparedStatement to prevent SQL injection. Below are the most important queries in the system.

Slot Conflict Check

```
SELECT COUNT(*) FROM appointments WHERE doctor_id=? AND appt_datetime=? AND status='SCHEDULED'
```

Patient Search

```
SELECT * FROM patients WHERE name LIKE ? OR phone LIKE ?
```

Appointment with Names

```
SELECT a.*, p.name AS pname, d.name AS dname FROM appointments a JOIN patients p ON  
a.patient_id=p.patient_id JOIN doctors d ON a.doctor_id=d.doctor_id
```

Admit — Occupancy Guard

```
UPDATE beds SET occupied=TRUE, patient_id=? WHERE bed_id=? AND occupied=FALSE
```

Payment Status Recalc

```
UPDATE invoices SET paid_amount=?, status=? WHERE invoice_id=?
```

Today's Appointments

```
SELECT ... FROM appointments WHERE DATE(appt_datetime) = CURDATE()
```

Dashboard Count

```
SELECT COUNT(*) FROM appointments WHERE status='SCHEDULED'
```

5. Extension & Maintenance Guide

5.1 Adding a New Module

- Define a new inner static class for the domain model with all fields, constructor, and toString()
- Add a CREATE TABLE IF NOT EXISTS statement in initDatabase()
- Write CRUD methods following the existing pattern (Scanner input → PreparedStatement → feedback)
- Create a *Menu(Scanner) method with a while(true) switch loop
- Add a new case in the main() switch to call the new menu

5.2 Upgrading to Spring Boot

The service methods map directly to Spring @Service components. Repository methods can be replaced with Spring Data JPA repositories. Menu methods become @RestController endpoints. The Singleton DB connection is replaced by Spring's DataSource auto-configuration.

5.3 Upgrading to a Connection Pool

Replace the getConnection() singleton with HikariCP for production deployments. Set a pool size of 5–10 for typical hospital workloads.

```
HikariConfig config = new HikariConfig();  
config.setJdbcUrl(DB_URL); config.setMaximumPoolSize(10);  
HikariDataSource ds = new HikariDataSource(config);
```

5.4 Known Limitations

Limitation	Recommended Fix
Single JDBC connection — not thread-safe	Replace with HikariCP connection pool
No authentication / login screen	Add users table with BCrypt password hashing
Console UI only	Wrap services in Spring Boot REST + React frontend
No audit log	Add created_at / updated_at timestamps and an audit_log table
DELETE may fail on FK	Add ON DELETE CASCADE or SET NULL to FK definitions
No pagination for large lists	Add LIMIT/OFFSET parameters to list methods